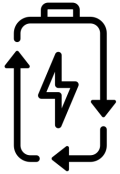
TDP Manager
A Planning and Reasoning project

Silvio d'Antonio - 2145048
Riccardo Mearelli - 1807331
Carlo Segreto - 2168340

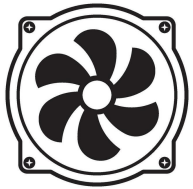
Planning and Reasoning course
Sapienza University of Rome

The domain

We propose an intelligent power management system for computers, which is able to schedule several given tasks to be performed while having battery and thermal constraints.
In particular it can decide whether to perform one of the assigned tasks with a low, medium, high or very high TDP (Thermal Design Power), trying to **maximize the overall performance** and not resulting in a state in which the battery is empty or in a throttling thermal condition.
We suppose that the system is also able to cool and recharge, both taking up some time, and that the user is not able to use the system during those special actions.



battery



cooling

The domain

Four tasks were taken into account. Each problem instance could have a maximum amount of hours in which all the assigned tasks must be completed. Every task has a minimum TDP requirement :

- **browsing**: low TDP,
- **working**: medium TDP,
- **entertainment**: medium TDP,
- **gaming**: high TDP.

A penalty is given based on how much the performance was lower than the maximum applicable.
The actions: **recharging** and **cooling** apply a penalty to the system as well.
Thus, the **goal** is to complete every task assigned, while ending in a state in which the battery is not drained and the system is not in thermal throttling. The metric is to **minimize the total-cost**, which corresponds to the total penalties applied by planned actions.

working

entertainment

gaming

browsing

PDDL: types and predicates

Types:

- battery
- task
- temp
- hour

Predicates:

- `(current ?h - hour)`
- `(next ?h1 - hour ?h2 - hour)`
- `(prec ?h2 - hour ?h1 - hour)`
- `(scheduled ?t - task ?h - hour)`
- `(battery-level ?b - battery)`
- `(battery-next ?b1 - battery ?b2 - battery)`
- `(temp-level ?t - temp)`
- `(temp-next ?t1 ?t2 - temp)`
- `(temp-prec ?t2 ?t1 - temp)`
- `(requires-medium ?t - task)`
- `(requires-high ?t - task)`

PDDL: actions

- do_low - perform one of the scheduled tasks with low TDP
- do_medium - perform one of the scheduled tasks with medium TDP
- do_high - perform one of the scheduled tasks with high TDP
- do_veryhigh - perform one of the scheduled tasks with very high TDP

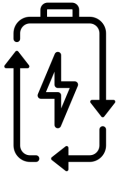
- cooling - reduce the temperature of the system
- recharge - recharge the battery of the system up to the max capacity

Action costs

Task	Low	Medium	High	Very High
Browsing	4	3	2	1
Working	-	3	2	1
Entertainment	-	3	2	1
Gaming	-	-	2	1
Cooling	5			
Recharge	6			

Easy schedule - definition

```
(scheduled working h1)
(scheduled entertainment h2)
(scheduled gaming h2)
(scheduled browsing h1)
(temp-level t20)
(battery-level b100)
```



100%

20°c

1h

2h

2h

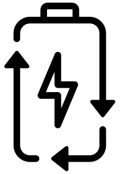
1h

Easy schedule - 6h duration

Search	Heuristic	Search time	Expanded nodes	Memory	Solution length	Solution cost
A*	blind	0.0188 s	463	41176 KB	6	11
	h_max	0.6540 s	345	56748 KB	6	11
	h_ff	0.5804 s	219	56748 KB	6	11
	h_add	0.7116 s	219	56748 KB	6	11
Greedy	h_max	0.5173 s	708	56752 KB	6	12
	h_ff	0.0433 s	12	56752 KB	6	12
	h_add	0.0422 s	12	56752 KB	6	12

Medium schedule - definition

```
(scheduled working h3)
(scheduled entertainment h2)
(scheduled gaming h2)
(scheduled browsing h2)
(temp-level t30)
(battery-level b80)
```



80%

30°c

3h

2h

2h

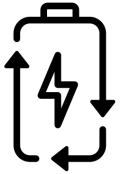
2h

Medium schedule - 9h duration

Search	Heuristic	Search time	Expanded nodes	Memory	Solution length	Solution cost
A*	blind	0.2793 s	30097	60792 KB	10	28
	h_max	38.8499 s	23381	86912 KB	10	28
	h_ff	46.8916 s	16548	86912 KB	10	28
	h_add	41.0724 s	14181	86912 KB	10	28
Greedy	h_max	3.0097 s	2092	86912 KB	11	34
	h_ff	0.2278 s	44	86912 KB	12	34
	h_add	0.1305 s	44	86912 KB	12	34

Hard schedule - definition

```
(scheduled working h3)
(scheduled entertainment h2)
(scheduled gaming h5)
(scheduled browsing h2)
(temp-level t40)
(battery-level b60)
```



60%

40°c

3h

2h

5h

2h

Hard schedule - 12h duration

Search	Heuristic	Search time	Expanded nodes	Memory	Solution length	Solution cost
A*	blind	0.9816 s	148678	67260 KB	15	45
	h_max	382.3299 s	118633	97548 KB	15	45
	h_ff	435.8991 s	90746	97548 KB	15	45
	h_add	428.8798 s	85622	97548 KB	15	47
Greedy	h_max	42.4522 s	15911	97548 KB	15	49
	h_ff	222.6774 s	55995	97548 KB	15	49
	h_add	165.2858 s	55753	97548 KB	15	49

Returned plan - Hard schedule.

A* Blind:

```
(do-high h1 h2 entertainment h2 h1 b60 b55 b50 b45 t40 t50)
(do-high h2 h3 entertainment h1 h0 b45 b40 b35 b30 t50 t60)
(do-high h3 h4 gaming h5 h4 b30 b25 b20 b15 t60 t70)
(do-high h4 h5 gaming h4 h3 b15 b10 b05 b00 t70 t80)
(recharge h5 h6 h7 t80 t90 b00)
(do-high h7 h8 gaming h3 h2 b100 b95 b90 b85 t90 t100)
(cooling h8 h9 t70 t80 t90 t100 t100)
(do-high h9 h10 gaming h2 h1 b85 b80 b75 b70 t70 t80)
(do-high h10 h11 gaming h1 h0 b70 b65 b60 b55 t80 t90)
(do-low h11 h12 browsing h2 h1 b55 b50 t80 t90 t100)
(do-high h12 h13 working h3 h2 b50 b45 b40 b35 t80 t90)
(do-low h13 h14 browsing h1 h0 b35 b30 t80 t90 t100)
(do-high h14 h15 working h2 h1 b30 b25 b20 b15 t80 t90)
(do-medium h15 h16 working h1 h0 b15 b10 b05 t90 t100)
(cooling h16 h17 t70 t80 t90 t100 t100)
cost = 45 (general cost)
```

PDDL Conclusion

- **A* blind:** it performs very well, despite the fact that no heuristic is available to the planner. This comes from the nature of the chosen problem, for which the heuristic computation overhead actually weighs more than the blind search.
- **A* h_max:** it performs pretty well, but it generally takes more time than A* blind even though the number of expanded nodes is almost the same, while there is a noticeable difference in terms of memory usage.
- **A* h_ff** & **h_add:** their performance was the worst overall, considering every combination of algorithm and heuristic. These, however, expand roughly half the nodes with respect to the other A* combinations.
- **Greedy h_max** & **h_ff** & **h_max:** it's a good compromise considering they take the same as an A* search in the worst case, therefore the solutions do not differ so much in terms of performance, while still being generally higher in terms of length and cost.

Note: experimenting with the cost associated with actions might significantly improve the running time, especially for non-blind A*.

IndiGolog

The domain was recalibrated to reconcile IndiGolog's reasoning about actions with practical execution times. This approach simplifies the problem instances maintaining the same goal and almost the same problem definition.
Simplified Problem Design :

- **Schedule reduction:** the hardest schedule considered it is of 9h in order to prevent the state-space explosion.
- **IDA* start:** to improve convergence and improve the execution time, the IDA algorithm was configured to start from a non-zero initial cost bound.
- **Ordered IDA* :** In order to further reduce the complexity and the execution time, the tasks can only be executed in a fixed order, one after the other. This limits the number of possible schedules without loss of generality.

IndiGolog - Fluents

```
task(browsing).
task(working).
task(entertainment).
task(gaming).
```

```
requires_medium(working).
requires_medium(entertainment).
requires_high(gaming).
```

```
fun_fluent(hour).
fun_fluent(temp).
fun_fluent(battery).
fun_fluent(totalCost).
fun_fluent(scheduled(_)).
```

Unlike the PDDL implementation where fluents are primarily boolean predicates, in inidiGolog they are treated as numeric fluents.

IndiGolog - Actions

```
prim_action(do_low(_)).

poss(do_low(K),and(task(K),and(neg(requires_medium(K))
and(neg(requires_high(K)),and(battery ≥ 5,
and(scheduled(K) > 0,hour ≤16)))))).

causes_val(do_low(K), hour, H2, H2 is hour + 1).
causes_val(do_low(K), scheduled(K), S2, S2 is scheduled(K) - 1).
causes_val(do_low(K), battery, B2, B2 is battery - 5).
causes_val(do_low(K), temp, T2, T2 is temp - 10).
causes_val(do_low(K), totalCost, C2, C2 is totalCost + 4).
```

```
prim_action(do_high(_)).

poss(do_high(K),and(task(K),and(battery ≥ 15,and(temp ≤ 90,
and(scheduled(K) > 0,hour ≤ 16))))).

causes_val(do_high(K), hour, H2, H2 is hour + 1).
causes_val(do_high(K), scheduled(K), S2, S2 is scheduled(K) - 1).
causes_val(do_high(K), battery, B2, B2 is battery - 15).
causes_val(do_high(K), temp, T2, T2 is temp + 10).
causes_val(do_high(K), totalCost, C2, C2 is totalCost + 2).
```

```
prim_action(do_medium(_)).

poss(do_medium(K),and(task(K),and(neg(requires_high(K)),
and(battery ≥ 10,and(temp ≤ 90,and(scheduled(K) >0,
hour ≤ 16)))))).

causes_val(do_medium(K), hour, H2, H2 is hour + 1).
causes_val(do_medium(K), scheduled(K), S2, S2 is scheduled(K) - 1).
causes_val(do_medium(K), battery, B2, B2 is battery - 10).
causes_val(do_medium(K), temp, T2, T2 is temp + 10).
causes_val(do_medium(K), totalCost, C2, C2 is totalCost + 3).
```

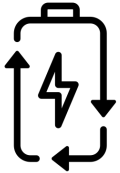
```
prim_action(do_veryhigh(_)).

poss(do_veryhigh(K),and(task(K),and(battery ≥ 20,
and(temp ≤ 80,and(scheduled(K) > 0,hour ≤ 16))))).

causes_val(do_veryhigh(K), hour, H2, H2 is hour + 1).
causes_val(do_veryhigh(K), scheduled(K), S2, S2 is scheduled(K) - 1).
causes_val(do_veryhigh(K), battery, B2, B2 is battery - 20).
causes_val(do_veryhigh(K), temp, T2, T2 is temp + 20).
causes_val(do_veryhigh(K), totalCost, C2, C2 is totalCost + 1).
```

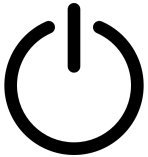
IndiGolog - Actions

```
prim_action(cooling).

poss(cooling,and(temp ≥ 100,hour ≤ 16)).

causes_val(cooling, hour, H2, H2 is hour + 1).
causes_val(cooling, temp, T2, T2 is temp - 30).
causes_val(cooling, totalCost, C2, C2 is totalCost + 5).
```

```
prim_action(turn_off).

poss(turn_off,and( battery≥5,and(hour ≤ 16,
and(scheduled(browsing) = 0,and(scheduled(working) = 0,
and(scheduled(entertainment) = 0,scheduled(gaming) = 0))))))).
```

```
prim_action(recharge).

poss(recharge,(and(battery ≤ 15,and(temp ≤ 90,hour ≤ 15)))).

causes_val(recharge, hour, H2, H2 is hour + 2).
causes_val(recharge, battery, 100, true).
causes_val(recharge, temp, T2, T2 is temp + 10).
causes_val(recharge, totalCost, C2, C2 is totalCost + 6).
```





IndiGolog - Exogenous Actions

```
exog_action(temp_up).
causes_val(temp_up, temp, T2, T2 is temp + 10).

exog_action(boil).
causes_val(boil, temp, 100, true).

exog_action(discharge).
causes_val(discharge, battery, B, B is battery - 10).

exog_action(idle).
causes_val(idle, hour, H, H is hour + 1).

exog_action(battery_swap).
causes_val(battery_swap, battery, 100, true).

exog_action(reset_stat).
causes_val(reset_stat, battery, 100, true).
causes_val(reset_stat, temp, 20, true).
```

Legality Task

For the legality task we consider a sequence of actions and want to check its executability from the initial situation. That means to check whether the preconditions for executing an action are satisfied according to the current situation.

Let's consider the following **initial situation**:

- Battery 80%
- Temperature 20°C
- Browsing 2h,
- Working 2h,
- Gaming 1h,

A possible **legal schedule** is:
ρ = do_veryhigh(gaming), do_medium(browsing), do_low(browsing), do_low(browsing)

A **non-legal schedule** is:
ρ = do_veryhigh(gaming), recharge
Cannot do recharge since the battery has not discharged enough yet.

```
# legal schedule: the plan is
# executed successfully

?- initialize(evaluator).
?- indigolog([do_veryhigh(gaming),
do_medium(browsing),
do_low(browsing), do_low(browsing)]).
```

```
# non-legal schedule: indigolog
# cannot execute the plan

?- initialize(evaluator).
?- indigolog([do_veryhigh(gaming),
recharge]).
```

Projection Task

For the projection task we consider a sequence of actions and want to study whether a fluent holds in the final situation.

Given the following **initial situation** (same as before):

- Battery 80%
- Temperature 20°C
- Browsing 2h,
- Working 2h,
- Gaming 1h,

And the following **sequence of actions**:
ρ = do_veryhigh(gaming), do_medium(browsing), do_low(browsing), do_low(browsing)

We can check that:

- Battery is not 100%
- Battery is instead 40%

```
# Battery is not at 100% after doing
# a few tasks

?- initialize(evaluator).
?- eval(battery == 100,
[do_veryhigh(gaming),
do_medium(browsing),
do_low(browsing), do_low(browsing)],
true).

# Battery is instead at 40% after
# doing a few tasks

?- initialize(evaluator).
?- eval(battery == 40,
[do_veryhigh(gaming),
do_medium(browsing),
do_low(browsing), do_low(browsing)],
true).
```

Controllers

DUMB

A *while* loop that non-deterministically picks through *ndet* any task and possible TDP trying to satisfy the goal.

IDA*

It reasons about actions through an exhaustive heuristic *search* to find the optimal history of tasks and TDP in order to minimize the total cost and reach the goal.

Ordered IDA*

Accelerates search by enforcing a task hierarchy which significantly prunes the search space without any loss in generality.

Ordered IDA* Reactive

Enhances the ordered version including an *interrupt* that triggers a new search whenever the goal is not met, allowing the agent to adapt to environment changes.

Returned histories (9h medium schedule)

Ordered IDA*

```
Program has executed to completion!!

History done:
[turn_off, do_high(gaming), cooling,
do_high(gaming), do_high(browsing), recharge,
do_low(browsing), do_high(entertainment),
do_high(entertainment), do_high(working),
do_high(working), do_high(working)]
```

-------- FINAL REPORT --------

Remaining battery:	55
Temperature level:	80
Current hour:	13
---	---
TOTAL COST:	31

Reactive ordered IDA*

```
Program has executed to completion!!

History done:
[stop_interrupts,
turn_off, do_high(gaming), do_high(gaming),
cooling, do_high(browsing), do_high(browsing),
recharge, discharge, cooling, discharge,
do_medium(entertainment),
do_high(entertainment), discharge,
do_high(working), cooling, boil,
do_high(working), do_high(working)]
```

-------- FINAL REPORT --------

Remaining battery:	40
Temperature level:	90
Current hour:	15
---	---
TOTAL COST:	40

Thank you for the attention!